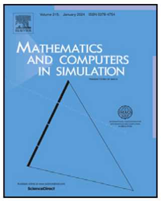
Mathematics and Computers in Simulation 238 (2025) 163–178

Contents lists available at ScienceDirect

Mathematics and Computers in Simulation

journal homepage: www.elsevier.com/locate/matcom



Original articles

Application of machine learning and convex limiting to subgrid flux modeling in the shallow-water equations

Ilya Timofeyev [a],[*], Alexey Schwarzmann [b], Dmitri Kuzmin [b]

[a] *Dept. of Mathematics, University of Houston, Houston, 77204, TX, USA*
[b] *Dept. of Mathematics, TU Dortmund, Dortmund, Germany*

ARTICLE INFO

Keywords:
Shallow water equations
Large eddy simulation
Subgrid scale modeling
Subgrid parametrization
Neural networks
Positivity preservation
Flux limiting

ABSTRACT

We propose a combination of machine learning and flux limiting for property-preserving subgrid scale modeling in the context of flux-limited finite volume methods for the one-dimensional shallow-water equations. The numerical fluxes of a conservative target scheme are fitted to the coarse-mesh averages of a monotone fine-grid discretization using a neural network to parametrize the subgrid scale components. To ensure positivity preservation and the validity of local maximum principles, we use a flux limiter that constrains the intermediate states of an equivalent fluctuation form to stay in a convex admissible set. The results of our numerical studies confirm that the proposed combination of machine learning with monolithic convex limiting produces meaningful closures even in scenarios for which the network was not trained.

1. Introduction

Many fluid flow models are based on nonlinear conservation laws that incorporate a vast range of spatial and temporal scales. Therefore, direct numerical simulations are often prohibitively expensive and, despite rapid progress in computer architecture, existing codes cannot be run at the discretization level required to resolve all relevant physical processes. Therefore, the derivation/calibration of reduced models has been an active area of research for many decades. The simplest Large-Eddy Simulation (LES) approaches use the Smagorinsky eddy viscosity model [1] for subgrid stresses that appear in the spatially filtered equations. More advanced alternatives include stochastic (e.g. [2–9]) and deterministic (e.g. [10–13]) closures, alpha models [14] etc. Recently, machine learning has also been used to represent the effects of unresolved degrees of freedom (e.g. [15–18], review [19]).

The main focus of our work is on the Shallow-Water Equations (SWE) [20]. This nonlinear hyperbolic system is widely used to simulate flow in rivers and channels, as well as coastal and ocean dynamics. In the context of ocean dynamics, parametrization of mesoscale eddies requires developing appropriate subgrid scale models. Recent years have witnessed the advent of closures based on Machine Learning (ML). The validity and effectiveness of such modern closures was demonstrated by many numerical examples. However, the use of Neural Network (NN) approximations makes it more difficult to prove the stability, consistency, and convergence of ML numerical methods for nonlinear PDEs. Moreover, for the SWE model, an NN-based closure may violate entropy inequalities or produce negative water heights. As a fail-safe remedy, we consider an NN model for representing subgrid fluxes (instead of estimating the whole right-hand side of the reduced model) and propose the application of a physics-aware flux limiter to ML-generated subgrid fluxes that may violate important inequality constraints. We use a relatively simple feed-forward network to extract subgrid fluxes from data generated by the fully resolved model. The monolithic convex limiting (MCL) strategy [21] that

---

[*] Corresponding author.

E-mail addresses: itimofey@cougarnet.uh.edu (I. Timofeyev), alexey.schwarzmann@math.tu-dortmund.de (A. Schwarzmann), kuzmin@math.uni-dortmund.de (D. Kuzmin).

https://doi.org/10.1016/j.matcom.2025.04.031
Received 24 July 2024; Received in revised form 14 January 2025; Accepted 26 April 2025

Available online 14 May 2025
0378-4754/© 2025 International Association for Mathematics and Computers in Simulation (IMACS). Published by Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

we use in this work represents the coarse-mesh cell averages as convex combinations of intermediate states that belong to a convex admissible set. The proofs of desired properties exploit this fact following the analysis of flux-corrected transport (FCT) algorithms by Guermond et al. [22].

The rest of the paper is organized as follows. We define the conserved variables and fluxes of the one-dimensional SWE system in Section 2. The space discretization of this nonlinear hyperbolic system and the definition of coarse-scale variables are discussed in Section 3. The fine-mesh discretization uses the Local Lax–Friedrichs (LLF) flux and is bound preserving. In Section 3.1, we construct the subgrid fluxes of the coarse-mesh model using a neural network. The MCL framework that we present in Section 3.2 is used for flux limiting purposes. In Section 4, we perform numerical experiments. The results of fully resolved simulations are compared with NN-based reduced model approximations. The effect of locally bound-preserving MCL corrections is studied by switching the flux limiter on and off. Conclusions are drawn in Section 5.

2. Shallow-water equations

We consider the one-dimensional SWE system with periodic boundary conditions imposed on the boundaries of the domain $\Omega = (0, L)$. The evolution of the fluid depth $h(x,t)$ and fluid velocity $v(x,t)$ is governed by

$$\begin{bmatrix} h \\ hv \end{bmatrix}_t + \begin{bmatrix} hv \\ hv^2 + \frac{1}{2}gh^2 \end{bmatrix}_x = 0, \tag{1}$$

where $x \in \bar{\Omega}$ is the space variable, $t \geq 0$ is the time variable, and g is the gravitational acceleration. The initial conditions are given by $h(x,0) = h_0(x)$, $v(x,0) = v_0(x)$. We assume that $h(x,t) > 0$, i.e., no dry states are present. Introducing the discharge $q = hv$, the hyperbolic system (1) can be rewritten as

$$\begin{bmatrix} h \\ q \end{bmatrix}_t + \begin{bmatrix} q \\ q^2/h + \frac{1}{2}gh^2 \end{bmatrix}_x = 0. \tag{2}$$

This is a system of the form $u_t + (f(u))_x = 0$, where $u = [h, q]^T$ is the vector of conserved quantities and $f(u) = [q, q^2/h + \frac{1}{2}gh^2]^T$ is the flux function. We initialize u by $u_0 = [h_0, q_0]^T$, where $q_0 = h_0 v_0$.

3. Discretization, coarse-scale variables, and subgrid fluxes

A finite-volume discretization of the SWE system on a uniform fine mesh with spacing $\Delta x = L/N_f$ produces a system of ordinary differential equations

$$\frac{\mathrm{d}}{\mathrm{d}t} u_i = \frac{F_{i-1/2} - F_{i+1/2}}{\Delta x} \qquad i = 1, \dots, N_f \tag{3}$$

for approximate fine-mesh cell averages $u_i(t) \approx \frac{1}{\Delta x} \int_{x_{i-1/2}}^{x_{i+1/2}} u(x,t)\mathrm{d}x$. The numerical flux

$$F_{i+1/2} = \frac{f(u_i) + f(u_{i+1})}{2} - \frac{\lambda_{i+1/2}}{2}(u_{i+1} - u_i) \tag{4}$$

of the Local Lax–Friedrichs (LLF) scheme is defined using the maximum wave speed $\lambda_{i+1/2}$ of the Riemann problem with the initial states u_i and u_{i+1}. The intermediate state

$$\bar{u}_{i+1/2} := \frac{u_{i+1} + u_i}{2} - \frac{1}{2\lambda_{i+1/2}}(f(u_{i+1}) - f(u_i)) \tag{5}$$

represents a spatially averaged exact solution of such a Riemann problem. In what follows, the state $\bar{u}_{i+1/2}$ is referred to as a *bar state*. Using the maximum eigenvalue of the Jacobian matrices $f'(u_i)$ and $f'(u_{i+1})$ as an approximate upper bound for the local wave speed, we set [23]

$$\lambda_{i+1/2} := \max\left(|v_i| + \sqrt{gh_i},\ |v_{i+1}| + \sqrt{gh_{i+1}}\right). \tag{6}$$

We denote by $\mathcal{F}(u_L, u_R)$ the numerical flux function such that $F_{i+1/2} = \mathcal{F}(u_i, u_{i+1})$ is given by (4) with $\lambda_{i+1/2}$ defined by (6). Substituting the so-defined LLF flux into (3) and invoking the definition (5) of $\bar{u}_{i+1/2}$, we write the spatial semi-discretization (3) in the equivalent *fluctuation form* (cf. [23])

$$\frac{\mathrm{d}}{\mathrm{d}t} u_i = \frac{1}{\Delta x}\left[\lambda_{i-1/2}(\bar{u}_{i-1/2} - u_i) + \lambda_{i+1/2}(\bar{u}_{i+1/2} - u_i)\right], \qquad i = 1, \dots, N_f. \tag{7}$$

In this paper, we utilize Heun's time-stepping method. This explicit two-step Runge–Kutta scheme is second-order accurate and yields a convex combination of two forward Euler predictors. Heun's method is a representative of *strong stability preserving* (SSP) methods [24–26]. The resulting space–time discretization also belongs to the class of *height positivity preserving* (HPP) methods, i.e., the fully discrete scheme guarantees that the water height $h_i(t)$ remains strictly positive for all times (see [27] for details).

We assume that the fine-mesh discretization resolves all physical processes of interest sufficiently well. This, essentially, implies that the number N_f of fine mesh cells

$$C_i = [x_{i-1/2}, x_{i+1/2}], \quad i = 1, \dots, N_f$$

is very large and the numerical viscosity introduced by the LLF scheme is very small. To avoid the high cost of direct numerical simulations on the fine mesh, we compose a coarse mesh from macrocells

$$\bar{C}_i := \bigcup_{j=k(i-1)+1}^{ki} C_j = [x_{k(i-1)+1/2}, x_{ki+1/2}], \qquad i = 1, \dots, N_c, \tag{8}$$

where k is the coarsening degree, $N_c = N_f/k$, and $\Delta X = k\Delta x$. The coarse-scale variables

$$U_i(t) = \frac{1}{k}\sum_{j=k(i-1)+1}^{ki} u_j(t), \qquad i = 1, \dots, N_c \tag{9}$$

correspond to averages over the macrocells $\bar{C}_i$. In principle, we could use the LLF finite volume discretization of the integral conservation law for $\bar{C}_i$ to directly obtain an evolution equation for U_i. However, a coarse-mesh LLF approximation is unlikely to resolve all physical processes appropriately even for moderate values of the parameter k. To derive *subgrid fluxes* that significantly reduce the error of the coarse-grid LLF scheme, we need to average Eqs. (7). The resulting equation for U_i can be written in the form

$$\frac{\mathrm{d}}{\mathrm{d}t}U_i = \frac{(\bar{F}_{i-1/2} + G_{i-1/2}) - (\bar{F}_{i+1/2} + G_{i+1/2})}{\Delta X}, \tag{10}$$

where $\bar{F}_{i+1/2} = \mathcal{F}(U_i, U_{i+1})$ is the coarse-mesh LLF flux. The fine-scale component of the subgrid flux

$$G_{i+1/2} := F_{ki+1/2} - \bar{F}_{i+1/2} \tag{11}$$

is given by $F_{ki+1/2} = \mathcal{F}(u_{ki}, u_{ki+1})$. The point $X_{i+1/2} = x_{ki+1/2}$ represents the interface of fine-mesh cells C_{ki} and C_{ki+1} such that $C_{ki} \cap C_{ki+1} = \bar{C}_i \cap \bar{C}_{i+1}$. The flux defined by (11) can be also expressed as

$$G_{i+1/2} := \mathcal{G}(u_{ki}, u_{ki+1}, U_i, U_{i+1}) = \mathcal{F}(u_{ki}, u_{ki+1}) - \mathcal{F}(U_i, U_{i+1}). \tag{12}$$

In general, the subgrid flux depends on the coarse-scale variables and the fine-mesh cell averages. The evolution Eq. (10) has the structure of a finite-volume discretization of the SWE system on a coarse mesh. The coarse-mesh LLF discretization without the subgrid fluxes is given by

$$\frac{\mathrm{d}}{\mathrm{d}t}U_i = \frac{\bar{F}_{i-1/2} - \bar{F}_{i+1/2}}{\Delta X}.$$

Since it is usually more diffusive than the fine-mesh discretization [23], the subgrid fluxes $G_{i+1/2}$ can be interpreted as antidiffusive corrections to the low-order flux approximations $\bar{F}_{i+1/2}$.

To obtain a closed-form equation for the coarse-scale variables, we need to construct a suitable approximation to the subgrid flux in terms of the coarse-mesh data. It is reasonable to expect that the subgrid flux $G_{i+1/2}$ depends on the coarse-mesh cell averages U_i and U_{i+1} but, in our experience, two-point stencils do not provide enough data for subgrid flux modeling using neural networks. Therefore, following Alcala [28], we use a larger stencil $[U_{i-1}, U_i, U_{i+1}, U_{i+2}]$ for the computation of the subgrid flux $G_{i+1/2}$. The coarse-mesh variables are used as input to the neural network that generates an approximation to $G_{i+1/2}$. The use of a larger computational stencil for this purpose can also be justified by the fact that the derivation of subgrid fluxes using the stochastic homogenization strategy [6,7,29] leads to extended-stencil approximations. Although the homogenization-based approach is valid in the limit of infinite separation of time scales between the coarse-scale components and fluctuations, a good agreement between the statistical properties of coarse-mesh variables in the fully resolved model and the reduced model was obtained in the stationary regime for the Burgers equation [6,29] and for the SWE [7]. Thus, we seek an approximation

$$G_{i+1/2} \approx \widetilde{\mathcal{G}}(U_{i-1}, U_i, U_{i+1}, U_{i+2}, \omega),$$

where $\widetilde{\mathcal{G}}$ is a nonlinear function represented by a neural network and ω is the vector of neural network's internal parameters (weights and biases).

3.1. Network architecture and training

In this paper, we use a feed-forward neural network as an approximating function $\widetilde{\mathcal{G}}$. We perform direct numerical simulations with the high-resolution model (3) to generate M data points

$$(U_{i-1}^{(j)}, U_i^{(j)}, U_{i+1}^{(j)}, U_{i+2}^{(j)}, G_{i+1/2}^{(j)}), \quad j = 1, \dots, M \tag{13}$$

and use the mean square loss function

$$C(\omega) := \frac{1}{M}\sum_{j=1}^{M}\left\|\widetilde{\mathcal{G}}(U_{i-1}^{(j)}, U_i^{(j)}, U_{i+1}^{(j)}, U_{i+2}^{(j)}, \omega) - G_{i+1/2}^{(j)}\right\|_2^2 \tag{14}$$

to train a neural network. It is important to note that, for any dataset $(U_{i-1}, U_i, U_{i+1}, U_{i+2})$, a fitted approximation to the subgrid flux $G_{i+1/2}$ is not necessarily unique. The "true" value (12) of $G_{i+1/2}$ depends on the subgrid states u_{ki} and u_{ki+1}, which are treated as unknown.

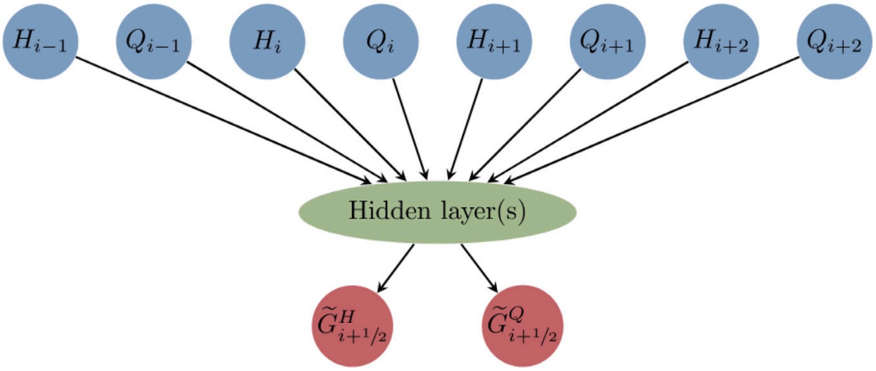


Fig. 1. Schematic representation of the neural network used to approximate $G_{i+1/2} = [G^H_{i+1/2}, G^Q_{i+1/2}]$.

Fig. 1 represents schematically the neural network used in our computations. The neural network has 3 hidden layers with N_{hid} neurons each. In this paper we consider $N_{hid} = 128, 64, 32, 16$ and discuss performance of these 4 neural networks for different parameter regimes. We use the leaky ReLU [30] activation function $f(z) = \max\{\alpha z, z\}$ with $\alpha = 0.01$. We also employ early stopping and data normalization as part of our regularization strategy. We use the default values of the `NeuralNet` class of the `skorch` package, unless stated otherwise. In particular, the default training algorithm `torch.optim.SGD` of the `PyTorch` package is used in our experiments.

For the training dataset, we generate n trajectories of the SWE using random initial conditions

$$h_0(x) = H_0 + A_h \sin(2\pi k_h x/L + \phi_h), \quad v_0(x) = V_0 + A_v \sin(2\pi k_v x/L + \phi_v). \tag{15}$$

The constant average height $H_0 = 2.0$ is used in all initial conditions for training. The remaining parameters are randomly generated following the distributions $V_0 \sim Unif(1,2)$, A_h, $A_v \sim Unif(0.2, 0.6)$, k_h, $k_v \sim Unif\{1,6\}$, ϕ_h, $\phi_v \sim Unif(0, 2\pi)$, where $Unif(a,b)$ denotes the continuous uniform distribution on (a,b) and $Unif\{n,m\}$ represents the discrete uniform distribution between $n, m \in N$. We simulate each trajectory until T^{train} and sample with the time step Δt. Specific values of n, T^{train}, and Δt are listed in Section 4.

Next, we run coarse-mesh simulations and compute the "true" subgrid fluxes (11). In each time step, we generate N_c subgrid fluxes $G_{i+1/2}$, $i = 1, \dots, N_c$. This computation generates $N_c \times n \times T^{train}/\Delta t$ data points for training the neural network with the batch size of 128. Initially, we train for 500 epochs with a learning rate of $\gamma = 0.1$ or $\gamma = 0.05$. The learning rate (in this range) for initial training does not seem to affect the quality of overall training at the end. Subsequently, we continue training for further 2000 epochs with a lower learning rate of $\gamma = 0.001$. As a regularization technique, we implement early stopping during both stages of training.

3.2. Flux limiting

One of the major drawbacks of machine learning is that there is a considerable gap in the theoretical understanding of neural networks. In our context, neural networks are treated as "black box" generators of subgrid fluxes $\widetilde{G}_{i+1/2}$. The addition of these fluxes to $\bar{F}_{i+1/2}$ can produce spurious oscillations and/or non-physical numerical solutions (e.g., negative water heights). To address this issue, we integrate the *monolithic convex limiting* (MCL) procedure [21] into the flux-corrected coarse-mesh discretization

$$\frac{\mathrm{d}}{\mathrm{d}t} U_i = \frac{(\bar{F}_{i-1/2} + \widetilde{G}^*_{i-1/2}) - (\bar{F}_{i+1/2} + \widetilde{G}^*_{i+1/2})}{\Delta X}, \tag{16}$$

where $\widetilde{G}^*_{i\pm1/2}$ denotes a constrained approximation to the network subgrid flux $\widetilde{G}_{i\pm1/2}$. To explain the MCL design philosophy, we write (16) in the equivalent bar state form

$$\frac{\mathrm{d}}{\mathrm{d}t} U_i = \frac{1}{\Delta X} \left[\Lambda_{i-1/2} (\bar{U}^{*,+}_{i-1/2} - U_i) + \Lambda_{i+1/2} (\bar{U}^{*,-}_{i+1/2} - U_i) \right], \tag{17}$$

where $\Lambda_{i+1/2}$ is the maximum wave speed depending on U_i and U_{i+1}. The low-order LLF part of

$$\bar{U}^{*,\pm}_{i+1/2} = \bar{U}_{i+1/2} \pm \frac{\widetilde{G}^*_{i+1/2}}{\Lambda_{i+1/2}} \tag{18}$$

is given by

$$\bar{U}_{i+1/2} := \frac{U_{i+1} + U_i}{2} - \frac{1}{2\Lambda_{i+1/2}} (f(U_{i+1}) - f(U_i)). \tag{19}$$

An *invariant domain* of a hyperbolic initial value problem is a convex admissible set containing all states that an exact solution may attain [31]. In the SWE context, a physical invariant domain $\mathcal{A}$ consists of all states $u = [h, hv]^T$ such that the water height h is nonnegative. The bar state $\bar{U}_{i+1/2}$ represents an averaged exact solution of the Riemann problem with the initial states U_i and

U_{i+1} [31,32]. Thus $\bar{U}_{i+1/2} \in \mathcal{A}$ provided that $U_i, U_{i+1} \in \mathcal{A}$. The MCL approach [33] ensures that $\bar{U}^{*,\mp}_{i\pm 1/2} \in \mathcal{A}$ whenever $\bar{U}_{i\pm 1/2} \in \mathcal{A}$. Each forward Euler stage of Heun's method advances the given data in time as follows:

$$U_i^{\mathrm{MCL}} = U_i + \frac{\Delta T}{\Delta X}\left[\Lambda_{i-1/2}(\bar{U}^{*,+}_{i-1/2} - U_i) + \Lambda_{i+1/2}(\bar{U}^{*,-}_{i+1/2} - U_i)\right]. \tag{20}$$

If the time step ΔT satisfies the CFL condition

$$\frac{\Delta T}{\Delta X}[\Lambda_{i+1/2} + \Lambda_{i-1/2}] \le 1, \tag{21}$$

then U_i^{MCL} is a convex combination of U_i and $\bar{U}^{*,\mp}_{i\pm 1/2}$. It follows that (cf. [31])

$$U_i, U_{i\pm 1} \in \mathcal{A} \quad \Rightarrow \quad U_i, \bar{U}_{i\pm 1/2} \in \mathcal{A} \quad \Rightarrow \quad U_i, \bar{U}^{*,\mp}_{i\pm 1/2} \in \mathcal{A} \quad \Rightarrow \quad U_i^{\mathrm{MCL}} \in \mathcal{A}.$$

Thus the MCL scheme is invariant domain preserving (positivity preserving) for time steps satisfying (21).

The above limiting strategy differs from classical flux-corrected transport (FCT) algorithms [34,35] and their extensions to hyperbolic systems [22,36,37] in that it does not split the fully discrete scheme (20) into a low-order predictor step and an antidiffusive corrector step. Instead, inequality constraints are imposed on the intermediate states $\bar{U}^{*,\mp}_{i\pm 1/2}$ of the spatial semi-discretization. In addition to constraining them to stay in the set $\mathcal{A}$ of physically admissible states, the MCL procedure can be configured to enforce semi-discrete entropy inequalities and/or local maximum principles for scalar quantities of interest [38].

The *sequential* MCL algorithm for the SWE system [33,39] imposes local bounds on h and v. In the present paper, we use a one-dimensional finite volume version of this algorithm to limit the fluxes

$$\widetilde{G}_{i+1/2} = [\widetilde{G}^h_{i+1/2}, \widetilde{G}^q_{i+1/2}]^T.$$

By abuse of notation, individual components of coarse-mesh variables are denoted by lowercase letters in the remainder of this section. For example, $\bar{U}_{i+1/2} = [\bar{h}_{i+1/2}, \bar{h}_{i+1/2}\bar{v}_{i+1/2}]^T$, where $\bar{v}_{i+1/2} := \bar{q}_{i+1/2}/\bar{h}_{i+1/2}$. Let

$$h_i^{\min} := \min(\bar{h}_{i-1/2}, \bar{h}_{i+1/2}), \qquad h_i^{\max} := \max(\bar{h}_{i-1/2}, \bar{h}_{i+1/2}), \tag{22}$$

$$v_i^{\min} := \min(\bar{v}_{i-1/2}, \bar{v}_{i+1/2}), \qquad v_i^{\max} := \max(\bar{v}_{i-1/2}, \bar{v}_{i+1/2}). \tag{23}$$

Following Hajduk [33], we formulate local maximum principles for the limited bar states as follows:

$$h_i^{\min} \le \bar{h}^{*,-}_{i+1/2} \le h_i^{\max}, \qquad h_{i+1}^{\min} \le \bar{h}^{*,+}_{i+1/2} \le h_{i+1}^{\max}, \tag{24a}$$

$$\bar{h}^{*,-}_{i+1/2} v_i^{\min} \le \bar{q}^{*,-}_{i+1/2} \le \bar{h}^{*,-}_{i+1/2} v_i^{\max}, \qquad \bar{h}^{*,+}_{i+1/2} v_{i+1}^{\min} \le \bar{q}^{*,+}_{i+1/2} \le \bar{h}^{*,+}_{i+1/2} v_{i+1}^{\max}. \tag{24b}$$

Since $\bar{h}^{*,\pm}_{i+1/2} = \bar{h}_{i+1/2} \pm \frac{\widetilde{G}^{h,*}_{i+1/2}}{\Lambda_{i+1/2}}$, the water height constraints (24a) are equivalent to

$$\Lambda_{i+1/2}(\bar{h}_{i+1/2} - h_i^{\max}) \le \widetilde{G}^{h,*}_{i+1/2} \le \Lambda_{i+1/2}(\bar{h}_{i+1/2} - h_i^{\min}), \tag{25}$$

$$\Lambda_{i+1/2}(h_{i+1}^{\min} - \bar{h}_{i+1/2}) \le \widetilde{G}^{h,*}_{i+1/2} \le \Lambda_{i+1/2}(h_{i+1}^{\max} - \bar{h}_{i+1/2}). \tag{26}$$

It follows that the best bound-preserving approximation to $\widetilde{G}^h_{i+1/2}$ is defined by

$$\widetilde{G}^{h,*}_{i+1/2} = \begin{cases} \min(\widetilde{G}^h_{i+1/2}, \Lambda_{i+1/2}\min(\bar{h}_{i+1/2} - h_i^{\min}, h_{i+1}^{\max} - \bar{h}_{i+1/2})) & \text{if } \widetilde{G}^h_{i+1/2} \ge 0, \\ \max(\widetilde{G}^h_{i+1/2}, \Lambda_{i+1/2}\max(\bar{h}_{i+1/2} - h_i^{\max}, h_{i+1}^{\min} - \bar{h}_{i+1/2})) & \text{otherwise.} \end{cases} \tag{27}$$

To ensure the feasibility of the discharge constraints (24b), the bar state

$$\bar{q}^{*,\pm}_{i+1/2} = \bar{q}_{i+1/2} \pm \frac{\widetilde{G}^{q,*}_{i+1/2}}{\Lambda_{i+1/2}} = \bar{h}^{*,\pm}_{i+1/2}\bar{v}_{i+1/2} \pm \frac{\Delta\widetilde{G}^{q,*}_{i+1/2}}{\Lambda_{i+1/2}}$$

is defined using a limited counterpart $\Delta\widetilde{G}^{q,*}_{i+1/2} = \widetilde{G}^{q,*}_{i+1/2} - \widetilde{G}^{h,*}_{i+1/2}\bar{v}_{i+1/2}$ of the flux difference

$$\Delta\widetilde{G}^q_{i+1/2} = \widetilde{G}^q_{i+1/2} - \widetilde{G}^{h,*}_{i+1/2}\bar{v}_{i+1/2}.$$

The sequential MCL algorithm [33] calculates the limited differences $\Delta\widetilde{G}^{q,*}_{i+1/2}$ and sets

$$\widetilde{G}^{q,*}_{i+1/2} := \widetilde{G}^{h,*}_{i+1/2}\bar{v}_{i+1/2} + \Delta\widetilde{G}^{q,*}_{i+1/2}. \tag{28}$$

Similarly to the derivation of the water height limiter (27), we use the equivalent form

$$\Lambda_{i+1/2}\bar{h}^{*,-}_{i+1/2}(\bar{v}_{i+1/2} - v_i^{\max}) \le \Delta\widetilde{G}^{q,*}_{i+1/2} \le \Lambda_{i+1/2}\bar{h}^{*,-}_{i+1/2}(\bar{v}_{i+1/2} - v_i^{\min}), \tag{29}$$

$$\Lambda_{i+1/2}\bar{h}^{*,+}_{i+1/2}(v_{i+1}^{\min} - \bar{v}_{i+1/2}) \le \Delta\widetilde{G}^{q,*}_{i+1/2} \le \Lambda_{i+1/2}\bar{h}^{*,+}_{i+1/2}(v_{i+1}^{\max} - \bar{v}_{i+1/2}) \tag{30}$$

Table 1
Parameters of initial conditions (15) used for testing.

IC	H_0	A_h	k_h	φ_h	V_0	A_v	k_v	φ_v
1	2.0	0.2	3	0.0	1.0	0.0	0	0.0
2	2.0	0.45	4	2.78	1.1	0.5	3	4.5
3	2.0	0.45	4	2.78	1.1	0.5	4	4.5
4	2.0	0.2	3	0.0	1.1	0.1	2	0.0

of conditions (24b) to establish their validity for (28) with $\Delta\widetilde{G}^{q,*}_{i+1/2}$ defined by

$$\Delta\widetilde{G}^{q,*}_{i+1/2}=\begin{cases}\min(\Delta\widetilde{G}^{q}_{i+1/2},\Lambda_{i+1/2}\min(\bar{h}^{*,-}_{i+1/2}(\bar{v}_{i+1/2}-v^{\min}_{i}),\bar{h}^{*,+}_{i+1/2}(v^{\max}_{i+1}-\bar{v}_{i+1/2}))) & \text{if } \Delta\widetilde{G}^{q}_{i+1/2}\geq 0,\\ \max(\Delta\widetilde{G}^{q}_{i+1/2},\Lambda_{i+1/2}\max(\bar{h}^{*,-}_{i+1/2}(\bar{v}_{i+1/2}-v^{\max}_{i}),\bar{h}^{*,+}_{i+1/2}(v^{\min}_{i+1}-\bar{v}_{i+1/2}))) & \text{otherwise.}\end{cases} \tag{31}$$

The enforcement of (24) guarantees positivity preservation and numerical admissibility of approximate solutions to the one-dimensional SWE system. For a more detailed description of the sequential MCL algorithm, we refer the interested reader to Schwarzmann [27]. Other examples of modern flux correction schemes for nonlinear hyperbolic systems and, in particular, for SWEs can be found in [22,36,40–42].

4. Numerical results

Considerable effort has been made to develop NN-based numerical approximations that are consistent with the physics of underlying PDE models, but the analysis of the resulting numerical algorithms can be difficult. Thus, although deep learning has become extremely popular in the context of computational methods for PDEs, numerical schemes based on neural networks can produce non-physical solutions. In the approach that we proposed above, subgrid fluxes generated by a neural network are "controlled" by a mathematically rigorous flux limiting formalism. The main goal here is to demonstrate that flux limiting can improve the quality of a deficient subgrid model by eliminating non-physical oscillations.

In our fine-mesh simulations for the SWE system (2) with $g = 9.812$, we discretize the computational domain $\Omega = (0, 100)$ using $N_f = 2000$ cells. As before, we assume that the fine-mesh discretization resolves all physical processes of interest properly. In particular, the numerical diffusion is negligible and further refinement does not produce any visible changes in the numerical solutions. We perform fine-mesh simulations using the time step $\Delta t = 0.1\Delta x$. The IDP property of the LLF space discretization and the SSP property of Heun's time-stepping method guarantee positivity preservation for this choice of Δt. Using the coarsening parameter $k = 20$, we construct coarse-mesh discretizations with $N_c = 100$ and $\Delta X = 1$. The time step $\Delta T = 0.005$ of the reduced model is an integer multiple of Δt. We also investigate the performance of reduced coarse models integrated numerically with larger time-steps $\Delta T = 0.0$ and $\Delta T = 0.1$. We consider the NN subgrid model with $N_{hid} = 128$ first, and then compare the performance of this NN with NNs with $N_{hid} = 64, 32, 16$ in a later paragraph. Initial conditions for the testing dataset are generated using Eq. (15) with parameter values that are not present in the training dataset. Selected parameters of initial conditions used for testing are presented in Table 1. The software used in this paper is available on GitHub [43].

We tested many initial conditions in this study. The overall performance of our subgrid model is discussed at the end of this section. We refer to simulations on the fine mesh as *full model* experiments or DNS (Direct Numerical Simulation). The results of reduced model experiments are referred to as *NN-reduced* if no MCL flux correction is performed and as *MCL-NN-reduced* if the MCL fix is activated.

A visual comparison of numerical solutions to the SWE obtained with $N_f = 2000$ and $N_c = N_f/k$ confirms that the subgrid fluxes $G_{i+1/2}$ are antidiffusive (not shown here; see [27]). Therefore, subgrid fluxes should "remove" additional diffusion resulting from the coarse-mesh discretization. However, this process is nonlinear and quite delicate. Since there is no control over the numerical performance of the neural network used to model subgrid fluxes, the NN-reduced model might produce non-physical solutions.

Several networks with the same configuration but with different amounts of training data are tested in this study. The amount of training data is determined by two parameters: (i) n, how many different trajectories are generated for training and (ii) T^{train}, the length of each trajectory. Each trajectory is sampled with the time step $\Delta t = 0.1\Delta x$. Given a lot of training data, the network typically reproduces fluxes quite well and the reduced model performs much better. However, in practical situations, the amount of training data can be limited since it is impossible to know a priori how many trajectories need to be generated for training.

First, we present results for a relatively large amount of training data with $n = 200$ and $T^{train} = 80$. Figs. 2 and 3 depict the discharge $q = hv$ and water height h at time $t = 200$ for several choices of initial conditions in simulations with the full model and both reduced models (without and with limiting).

In this example, the application of MCL to the subgrid fluxes does not significantly affect the quality of the results. It is worth pointing out that we test the performance of reduced models on a much longer interval compared to the length of training trajectories. Specifically, the length of the testing trajectory is $T^{test} = 200$, while the length of each training trajectory is $T^{train} = 80$. The performance of reduced models deteriorates for larger times and solutions with relatively small waves, but our ML reduced models can predict the dynamics of the SWE with larger waves for a much longer time. Numerical solutions for $T^{test} = 400$ are compared in Fig. 4.

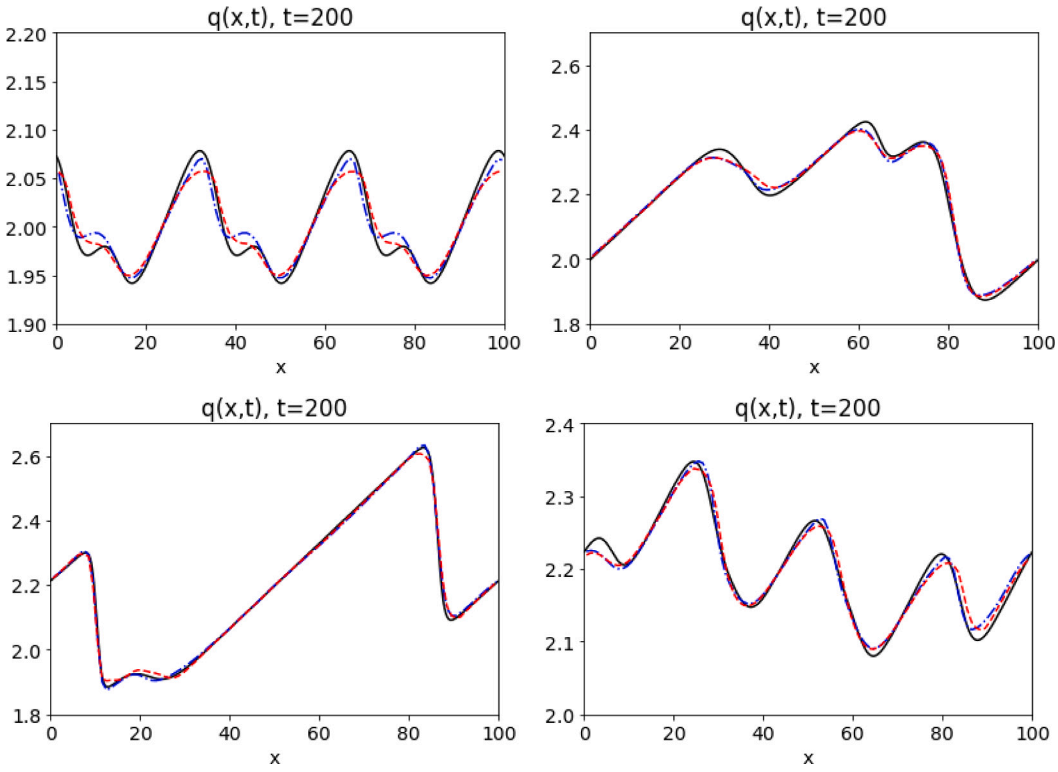


Fig. 2. Discharge $q(x,t) = hv$ at time $t = 200$ in simulations of the full model (black solid line) with $\Delta x = 0.05$ and reduced models with $k = 20$ ($\Delta x = 1$) and NN with $N_{hid} = 128$ trained with $n = 200$ and $T = 80$; Blue Dash-Dot Line — NN-reduced model, Red Dashed Line — MCL-NN-reduced model. Initial conditions with parameters in Table 1 (1 — top-left, 2 — top-right, 3 — bottom-left, 4 — bottom-right). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

The performance of ML reduced models changes drastically when the network is trained on a smaller dataset. For instance, we also generated the training dataset with $n = 100$ and $T^{train} = 40$. Thus, the training dataset is four times smaller compared to the training dataset for the reduced model used above. A comparison of the discharge at time $T = 200$ for the DNS and simulations with the two reduced models (without and with the MCL fix) is presented in Fig. 5. The reduced model without the limiter produces spurious oscillations in all four cases. These oscillations are particularly evident in the top-left subplot. The MCL-NN reduced model corrects these small-scale oscillations and produces physical solutions.

One can use the MCL limiter as an indicator of the training quality for the NN reduced model. The MCL limiter is applied after training and the MCL-NN reduced model has approximately the same computational cost as the NN reduced model without the limiter. Therefore, if the two models produce very similar results, then this implies that the neural network approximates the fluxes with very good accuracy and is unlikely to produce non-physical solution values. However, if the two reduced models result in considerably different numerical solutions, then this can be an indicator that the neural network is not trained sufficiently well.

Comparison with Traditional Subgrid Models. We also compared our subgrid closure with more traditional approaches for modeling the subgrid terms. In particular, the application of a flux limiter to a coarse-mesh discretization in conjunction with a high-order flux approximation can be interpreted as an implicit subgrid scale model for “monotonically integrated” Large Eddy Simulation (MILES); see, e.g., [44–46]. Thus, we compare our subgrid NN closure with MCL-constrained results for target fluxes corresponding to the Lax–Wendroff closure.

Short simulations reveal that the MCL-Lax–Wendroff (MCL-LW) scheme is much more diffusive compared to our MCL-NN closure model. Numerical results comparing the two subgrid models are depicted in Fig. 6. This figure demonstrates that the MCL-LW model with $N_x = 100$ ($\Delta x = 1$) is very diffusive and does not represent the accurate numerical solution at all. The MCL-LW model with $N_x = 500$ ($\Delta x = 0.2$) is also diffusive and leads to incorrect (higher) minima and considerably less sharp fronts in numerical solutions. Our MCL-NN subgrid model with $N_x = 100$ ($\Delta x = 1$) correctly represents the numerical solution in resolved simulations. We also verified these results with different initial conditions and longer simulations times. They are not presented here only for the brevity of the presentation.

Simulations with MCL-NN Subgrid Model using Different Time-Steps. The CFL requirement for the fully resolved model appears to be approximately $\Delta t = 0.1\Delta x$ where $\Delta x = L/N_f = 0.05$. The upper bound on the wave propagation is given by (6), and $\sqrt{gH_0} \approx 4.4$. Initially, $V_0 \approx 1$ and the upper bound on the speed of wave propagation can be roughly estimated as $|V_0| + \sqrt{gH_0} \approx 5.4$. Thus, the LLF model with $N_f = 2000$ and $\Delta t = 0.2\Delta x$ violates the CFL condition and produces non-physical oscillations (not shown here).

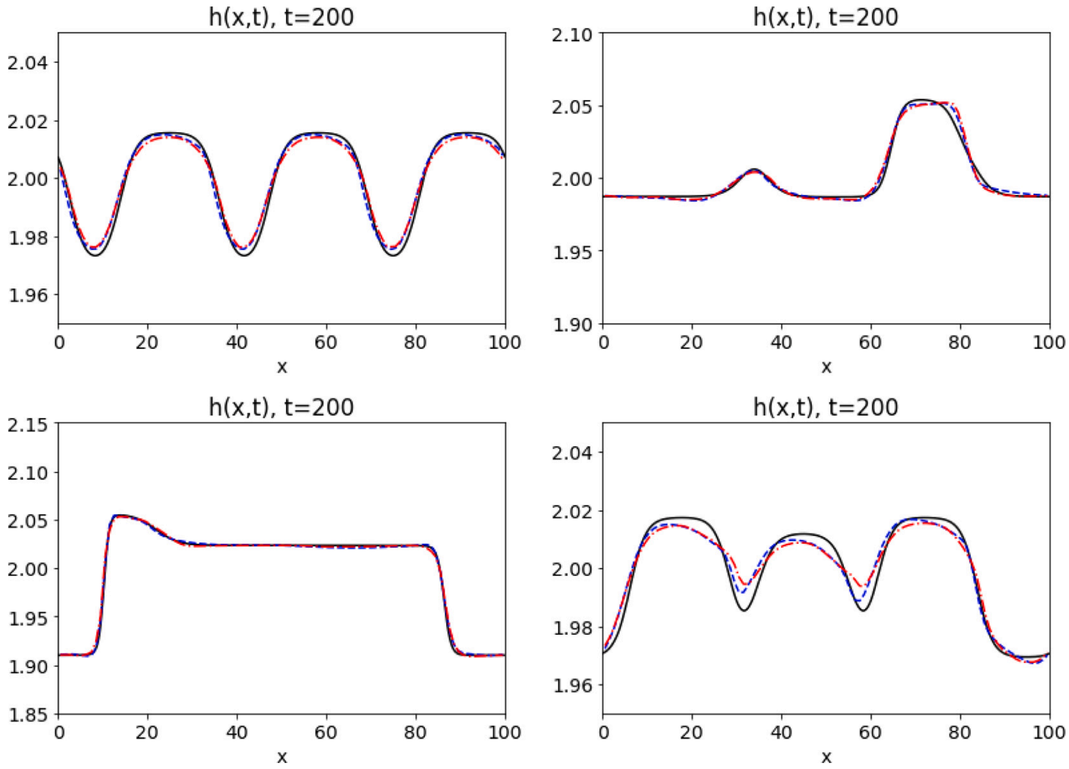


Fig. 3. Water height $h(x,t)$ at time $t = 200$ in simulations of the full model (black solid line) with $\Delta x = 0.05$ and reduced models with $k = 20$ ($\Delta x = 1$) and NN with $N_{hid} = 128$ trained with $n = 200$ and $T = 80$; Blue Dash-Dot Line — NN-reduced model, Red Dashed Line — MCL-NN-reduced model. Initial conditions with parameters in Table 1 (same location as in Fig. 2). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

The main advantage of the coarse model is that it allows using a much larger time-step, since the mesh size is significantly larger than in the fully resolved simulations. Figs. 7 and 8 depict a comparison of simulations for MCL-NN models integrated with several different time-steps $\Delta t = 0.005$, 0.05, 0.1. Recall that the time-step of the fully resolved LLF model is $\Delta t = 0.005$. There are small discrepancies between MCL-NN reduced models integrated with different time-steps, but numerical results with three different time-steps are in a very good agreement with fully resolved simulations. Thus, coarser mesh simulations with neural networks allow for additional acceleration due to a much larger time-step compared with the fully resolved model.

Simulations with Networks of Different Sizes with the Same Training Data. Since accelerated simulations are the main motivation for introducing subgrid models, we investigate the performance of neural networks with decreasing number of neurons in hidden layers. The overall architecture of neural networks remains the same (i.e. 3 hidden layers) as discussed in Section 3.1, but we consider NNs with $N_{hid} = 128$, 64, 32, 16 neurons in each hidden layer. Recall that the neural network discussed previously has 3 hidden layers with $N_{hid} = 128$ neurons in each layer. We train all 4 neural networks on the same data set with $n = 200$ and $T^{train} = 80$. In this paragraph, we describe the performance of 4 neural networks in the regime that is close to the training data. In particular, we consider initial conditions such that $H_0 \approx 2$ and $V_0 \approx 1$. We also use the MCL limiter in all simulations (we refer to these closure models as MCL-NN). Fig. 9 depicts a comparison of the water heights $h(x,t)$ for the fully resolved simulations and simulations with 4 neural networks using the time-step $\Delta t = 0.05$. Velocity and discharge follow a similar trend. Overall, the performance of 3 neural networks with $N_{hid} = 128, 64, 32$ is very good. In fact, the performance of MCL-NN with $N_{hid} = 32$ is sometimes better than that of MCL-NN with $N_{hid} = 128$. This might occur because the neural network with $N_{hid} = 128$ has too many parameters and is overfitted. The training loss is increasing with N_{hid}. In particular, at the end of training, the training loss for $N_{hid} = 128, 64, 32, 16$ is $loss = 1.5, 1.8, 2.7, 6.4\,(\times 10^{-4})$, respectively. It is noticeable that the performance of MCL-NN with $N_{hid} = 16$ deteriorates for longer times $t > 150$. The performance of MCL-NN with $N_{hid} = 16$ is comparable with other networks for times $t < 100$. However, using NN with $N_{hid} = 16$ without the MCL sometimes leads to non-physical oscillatory solutions. Thus, the MCL limiter becomes essential for correcting these oscillations, and simulations of the MCL-NN subgrid model agree with the resolved simulations reasonably well. The oscillatory behavior of NN with $N_{hid} = 16$ and corrected MCL-NN with $N_{hid} = 16$ is depicted in Fig. 10.

We can also observe that all four MCL-NN subgrid models produce small low-frequency oscillations near fronts with high gradients. This is particularly visible for shorter times (e.g. Fig. 10). However, due to the MCL limiter, these oscillations do not grow in time, and the coarse mesh solutions agree with the results of fully-resolved simulations very well. In addition, we also

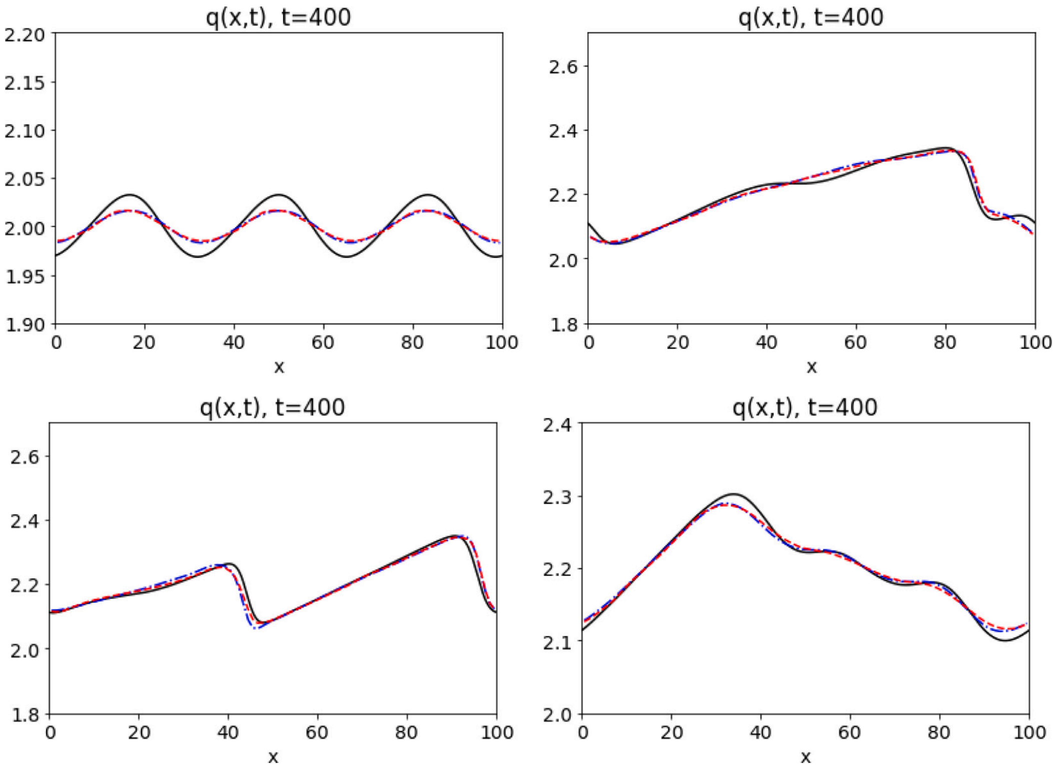


Fig. 4. Discharge $q(x,t) = hv$ at time $t = 400$ in simulations of the full model (black solid line) with $\Delta x = 0.05$ and reduced models with $k = 20$ ($\Delta x = 1$) and NN with $N_{hid} = 128$ trained with $n = 200$ and $T = 80$; Blue Dash-Dot Line — NN-reduced model, Red Dashed Line — MCL-NN-reduced model. Initial conditions with parameters in Table 1 (same location as in Fig. 2). Notice the difference in the vertical scale between the upper left subplot and the other three subplots. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

observe that the subgrid MCL-NN model with $N_{hid} = 32$ performs slightly better than other models. Running times are presented in Appendix. PyTorch automatically parallelizes simulations with MCL-NN128 and MCL-NN64. Running times reported in the appendix take the CPU utilization into account and are equivalent to a single-CPU simulation. Parallelized NN simulations are considerably shorter (in real time) than a single-CPU fully resolved simulation. However, parallelization might introduce a considerable additional overhead; thus running times presented here serve only as a rough guideline.

Simulations with Networks of Different Sizes outside of the Training Regime. Figs. 11 and 12 depict simulation results for reduced MCL-NN models with $N_{hid} = 128, 64, 32, 16$ outside of the training regime. In particular, all four subgrid models are trained in the regime $H_0 = 2$ and then tested with the averaged water height $H_0 = 1.5$ and $H_0 = 2.5$. This is a ±25% fluctuation of H_0 compared to the training regime. Here we present results for only one initial condition and two values of H_0, but we also tested on other initial conditions and values of parameters with comparable results. All four subgrid models perform well outside of the training regime, except, possibly, the MCL-NN model with $N_{hid} = 16$ for $H_0 = 1.5$. Moreover, the MCL-NN with $N_{hid} = 32$ performs slightly better than the MCL-NN subgrid model with $N_{hid} = 128$ and the MCL-NN subgrid model with $N_{hid} = 64$ produces the best agreement with the fully-resolved simulations overall. Performance of all four MCL-NN models deteriorates for values of H_0 outside of the ±25% range. The performance of MCL-NN subgrid models declines faster for smaller values of H_0. There is still a reasonably good agreement between coarse models and fully resolved simulations for larger values of $H_0 \in [2.5, \ldots, 3]$. We obtained similar results for simulations using coarse models with a different averaged velocity in the range $V_0 \in [0.6, \ldots, 1.4]$. Performance of subgrid MCL-NN models is less sensitive to fluctuations in V_0, probably because V_0 is not a conserved quantity. Moreover, the training dataset is generated with $V_0 \in [1, 2]$. Thus, V_0 has much larger (relative) fluctuations in the training dataset compared to H_0. Overall, MCL-NN subgrid models with $N_{hid} = 128$, 64, 32 perform very well outside of the training regime for a wide range of parameters H_0 and V_0. We also observe that in tests with varying H_0 and V_0, the MCL-NN subgrid models with $N_{hid} = 64, 32$ perform comparably to the MCL-NN subgrid model with $N_{hid} = 128$. This is another indication that the computational complexity of MCL-NN subgrid models can be reduced significantly compared to the computational cost of NN with $N_{hid} = 128$.

Simulations with Non-Smooth Initial Conditions. We also consider the initial condition

$$h(x,0) = \begin{cases} H_0 + a & \frac{L}{3} < x < \frac{2L}{3}, \\ H_0 - a/2 & \text{otherwise}, \end{cases} \qquad v(x,0) = 0, \tag{32}$$

in which H_0 represents the averaged water height, and a controls the magnitude of the “step”. In particular, we tested the performance of subgrid models discussed in previous sections with $H_0 = 1.6$, 2, 2.4 and $a = 0.5$, 0.75 for the initial condition given

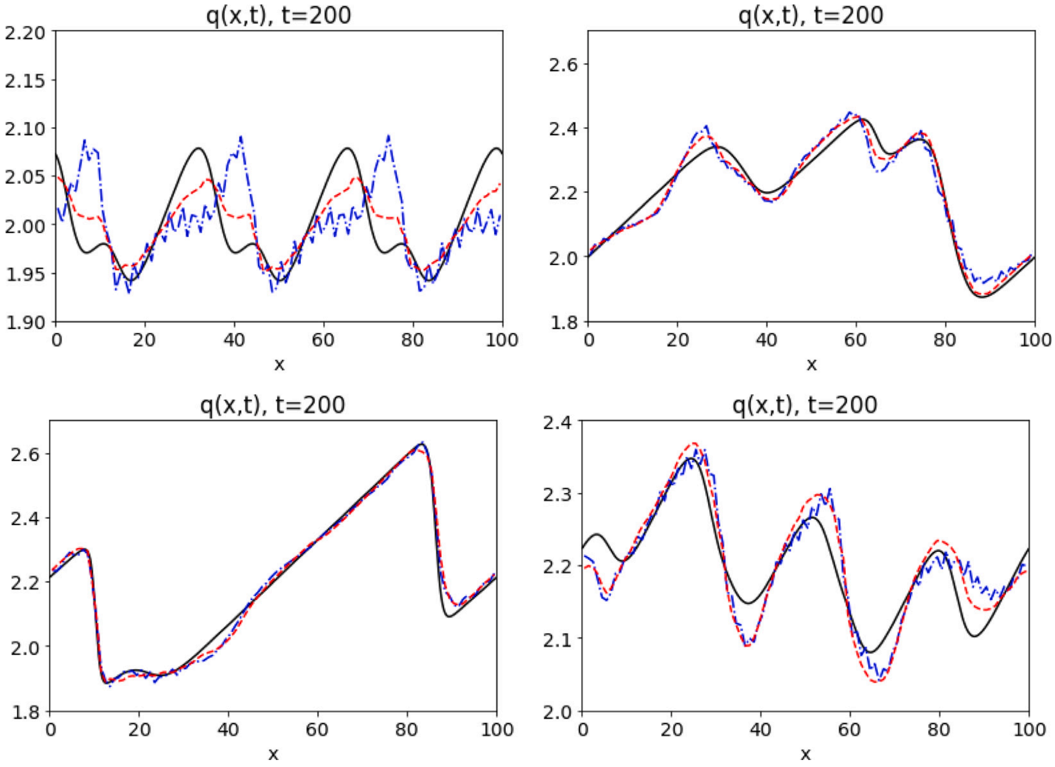


Fig. 5. Discharge $q(x,t) = hv$ at time $t = 200$ in simulations of the full model (black solid line) with $\Delta x = 0.05$ and reduced models with $k = 20$ ($\Delta x = 1$) and NN with $N_{hid} = 128$ trained with $n = 100$ and $T^{train} = 40$; Blue Dash-Dot Line — NN-reduced model, Red Dashed Line — MCL-NN-reduced model. Initial conditions with parameters in Table 1 (same location as in Fig. 2). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

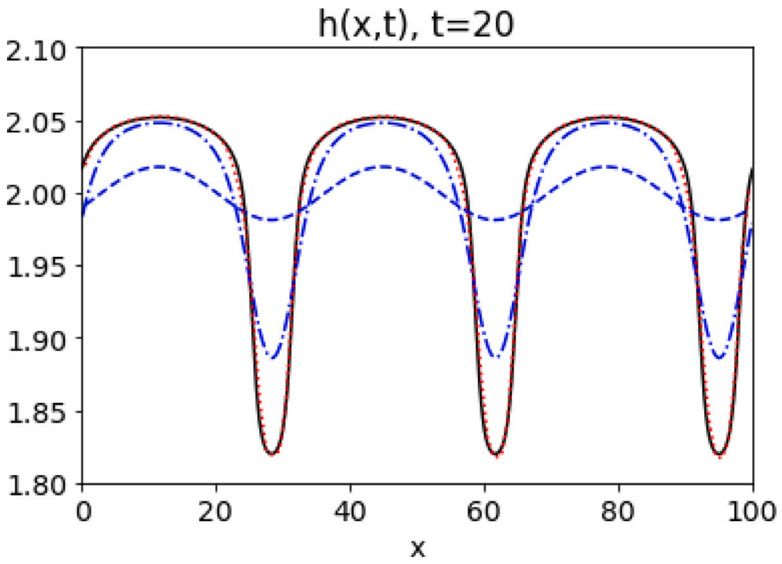


Fig. 6. Water height $h(x,t)$ at time $t = 20$ in simulations of the full model (black solid line) with $N_x = 2000$ ($\Delta x = 0.05$), MCL-LW models with $N_{hid} = 128$, $N_x = 100$ ($\Delta x = 1$) and $N_x = 500$ ($\Delta x = 0.2$) (blue dashed and blue dash-dot lines, respectively), and MCL-NN reduced models with $N_x = 100$ ($\Delta x = 1$) trained with $n = 200$ and $T^{train} = 80$ (red dotted line nearly overlaps with the black line). Initial conditions #1 in Table 1. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

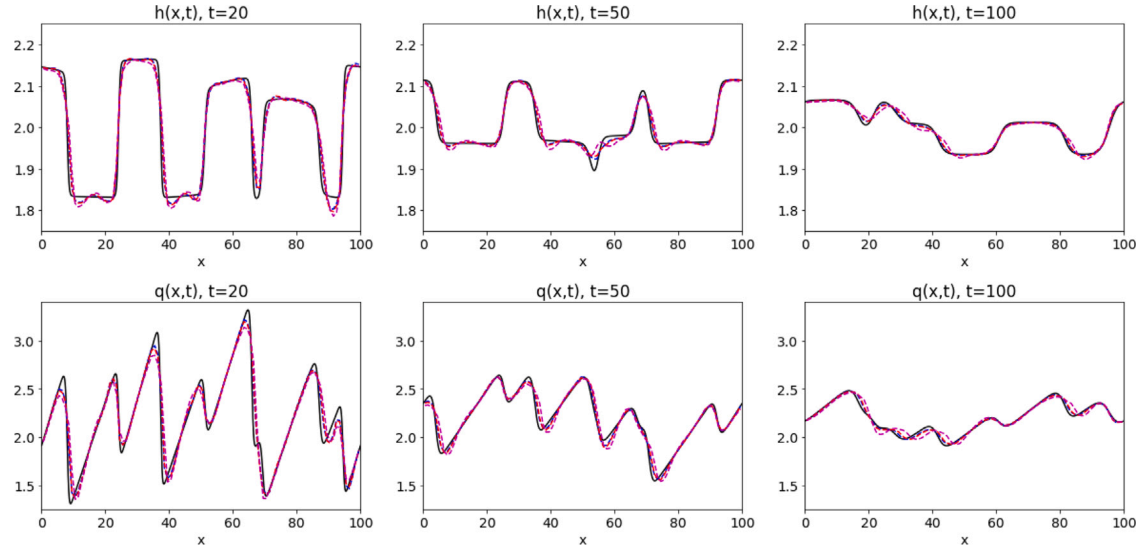


Fig. 7. Water height $h(x,t)$ and discharge $q(x,t)$ at time t = 20, 50, 100 in simulations of the full model (black solid line) and MCL-NN reduced models with $N_{hid} = 128$, $N_x = 100$ ($\Delta x = 1$) trained with $n = 200$ and $T^{train} = 80$ with time-step $\Delta t = 0.005$, 0.05, 0.1 (dashed blue, red, and magenta lines, respectively). Initial conditions #2 in Table 1. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

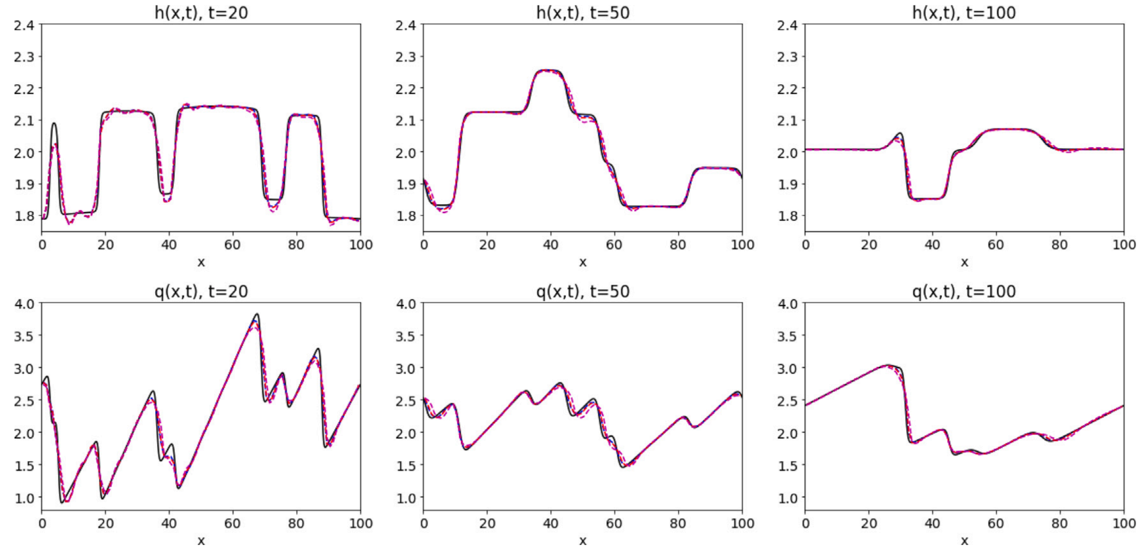


Fig. 8. Water height $h(x,t)$ and discharge $q(x,t)$ at time t = 20, 50, 100 in simulations of the full model (black solid line) and MCL-NN reduced models with $N_{hid} = 128$, $N_x = 100$ ($\Delta x = 1$) trained with $n = 200$ and $T^{train} = 80$ with time-step $\Delta t = 0.005$, 0.05, 0.1 (dashed blue, red, and magenta lines, respectively). Initial conditions #3 in Table 1. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

by (32). Similarly to other simulations presented earlier, we tested subgrid NN models with $N_{hid} = 128, 64, 32, 16$ trained on $n = 200$ and $T^{train} = 80$. We would like to point out that all subgrid NN models are trained on smooth data with $H_0 = 2$. The MCL limiter is crucial in simulations with non-smooth initial conditions. Without the limiter, all subgrid NN models quickly develop high-frequency spurious oscillations. The MCL limiter nearly eliminates these oscillations. Numerical results for the water height in simulations with initial (32) with $H_0 = 2.4$ and $a = 0.5$ for subgrid models with $N_{hid} = 128$ and $N_{hid} = 32$ are presented in Fig. 13. We obtained similar results for subgrid models with $N_{hid} = 64$ and $N_{hid} = 16$. We would like to note that numerical all solutions of coarse models have small oscillations, but these oscillations do not grow in time due to the MCL. Overall, these results are consistent with our intuition from numerical analysis — the NN parametrization represents a high-order flux discretization (or a high-order correction to flux parametrization) that cannot be applied to non-smooth initial conditions without a flux limiter. In addition, recalling our comparison between the MCL-Lax–Wendroff and MCL-NN parametrizations discussed earlier, the NN parametrization represents flux discretization of the of a higher-order compared to the LW method, since the MCL-NN parametrization reproduces sharp front with high accuracy compared to the MCL-LW simulations with the same resolution.

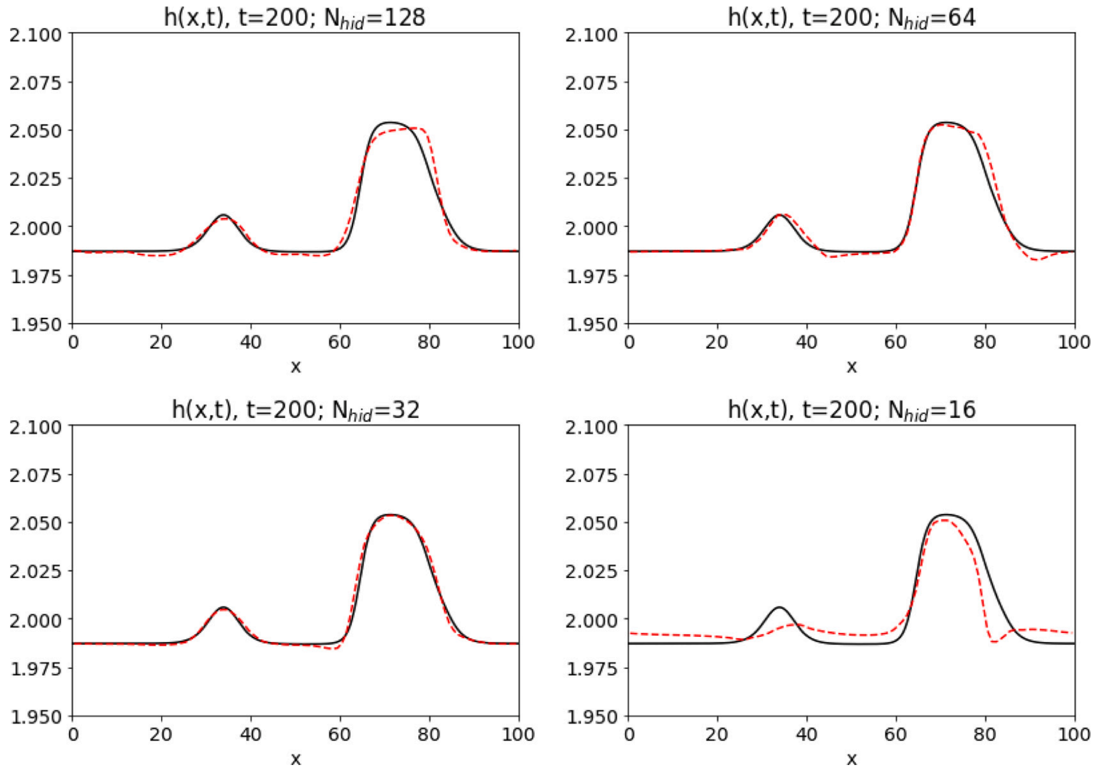


Fig. 9. Water height $h(x,t)$ at time $t = 200$ in simulations of the full model (black solid line) and MCL-NN reduced models with $N_x = 100$ ($\Delta x = 1$) trained with $n = 200$ and $T^{train} = 80$ with time-step $\Delta t = 0.05$ (red dashed line). Top left, top right, bottom left, bottom right — $N_{hid} = 128, 64, 32, 16$, respectively. Initial conditions #2 in Table 1. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

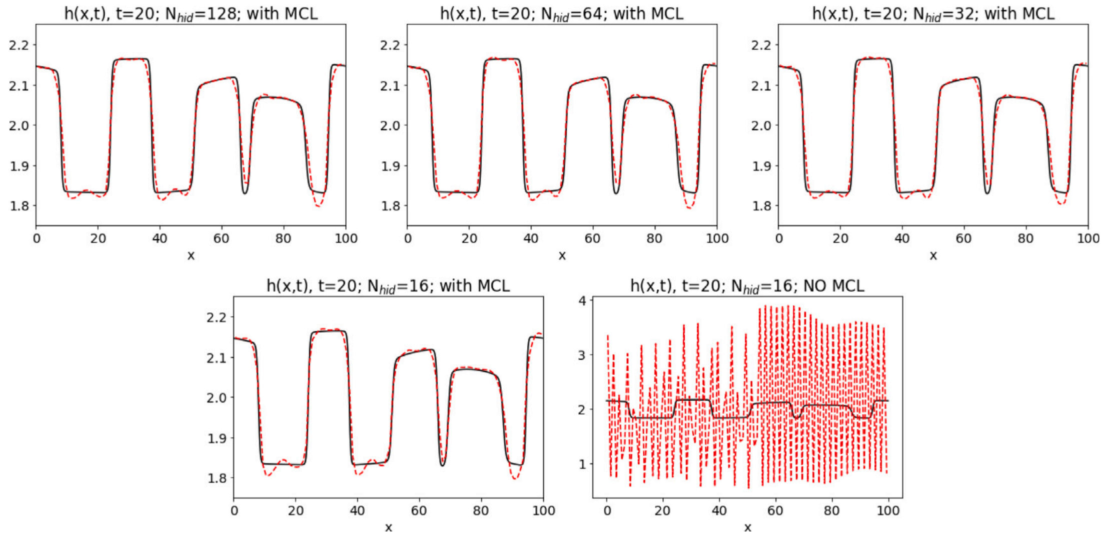


Fig. 10. Water height $h(x,t)$ at time $t = 20$ in simulations of the full model (black solid line) and NN reduced models with $N_x = 100$ ($\Delta x = 1$) trained with $n = 200$ and $T^{train} = 80$ with time-step $\Delta t = 0.05$ (red dashed line). Top row — $N_{hid} = 128, 64, 32$ with the MCL. Bottom left — $N_{hid} = 16$ with the MCL, bottom right — $N_{hid} = 16$ without the MCL. Initial conditions #2 in Table 1. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

5. Conclusions

In this work, we developed a neural network reduced model for coarse-grained simulations using the shallow water equations. Starting with a fine-mesh finite-volume discretization, we defined the coarse-mesh variables as “box” spatial averages. Then all

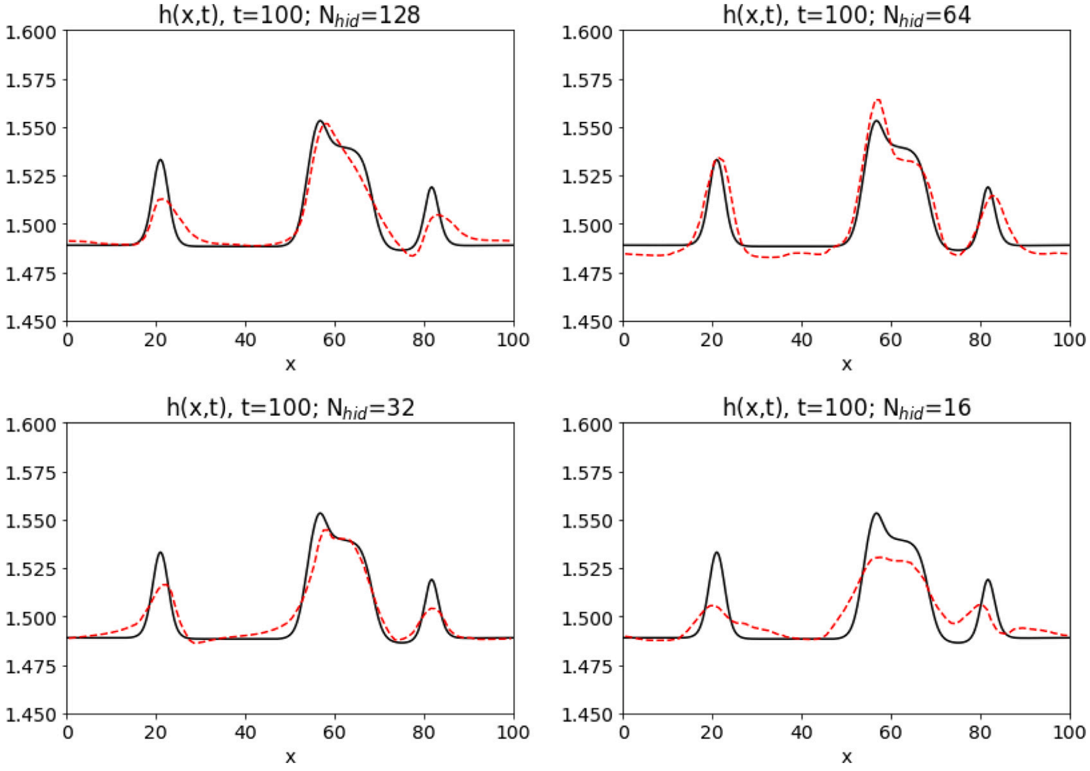


Fig. 11. Water height $h(x,t)$ at time $t = 100$ in simulations of the full model (black solid line) with $H_0 = 1.5$ and MCL-NN reduced models with $N_x = 100$ ($\Delta x = 1$) trained on $H_0 = 2$ with $n = 200$ and $T^{train} = 80$ with time-step $\Delta t = 0.05$ (red dashed line). Top left, top right, bottom left, bottom right — $N_{hid} = 128, 64, 32, 16$, respectively. Initial conditions with $H_0 = 1.5$ and other parameters as in #2 in Table 1. Compare with the top right plot in Fig. 7 where $H_0 = 2$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

dynamic variables were decomposed into coarse-scale components (i.e., discretization of the SWE on a coarse mesh) and fine-scale fluctuations. The reduced model uses a relatively simple feed-forward neural network to represent flux corrections due to the subgrid terms (fluctuations).

The neural network is initially trained on a large dataset generated using direct numerical simulations with the fine-mesh discretization of the SWE. Our numerical studies indicate that the reduced model is capable of capturing the dynamics of coarse-scale variables for a wide range of initial conditions and for a much longer time compared with the simulation time of each trajectory in the training dataset. In addition, our results indicate that the NN reduced model can be used outside of the training regime. In particular, we demonstrated good performance of NN subgrid models in regimes where the average water height is varied within ±25% compared to the training data. We would like to point out that the NN parametrization is local in space and requires only a four-point stencil. Local parametrizations have various practical advantages compared to global (e.g., Fourier space) parametrizations of unresolved degrees of freedom. For instance, our approach can be easily generalized to other types of boundary conditions and spatially non-homogeneous problems (e.g., with bottom topography). Moreover, in the spatially non-homogeneous case, neural networks can be trained in parallel using only local four-point stencil data. We will explore these issues in a subsequent paper for a more realistic ocean flow application. Finally, our neural network is relatively simple and has only three layers, thus, providing a minimal computational overhead. Moreover, the network architecture is fairly robust. We tested networks of different sizes and found that smaller networks with 32 neurons in each hidden layer yield very good results. We also demonstrated that the reduced model can be accelerated by using a much larger computational time-step compared to fully resolved simulations.

The advantage of the approach considered here is that we train a reduced model for subgrid fluxes instead of estimating the whole right-hand side directly. Therefore, the reduced NN model is amenable to more traditional numerical techniques, such as flux limiting. Thus, we apply the Monolithic Convex Limiting (MCL) strategy to ensure that the machine learning component of the reduced model does not produce non-physical fluxes. Overall, we believe that combining machine learning with existing tools from numerical analysis has many potential advantages, such as improving the forecasting capabilities of ML reduced models, the ability to use simpler neural networks with smaller computational overhead, developing real-time and a posteriori tools for verifying the quality of ML reduced models.

Using numerical simulations, we demonstrated that the flux limiter plays a minimal role when the NN model is well trained and utilized in the same regime as the training data (e.g. smooth data). This is not surprising, since the Neural Network is capable of learning complex functions and, thus, is able to reproduce subgrid fluxes and ensure that they (almost) lie within physical limits.

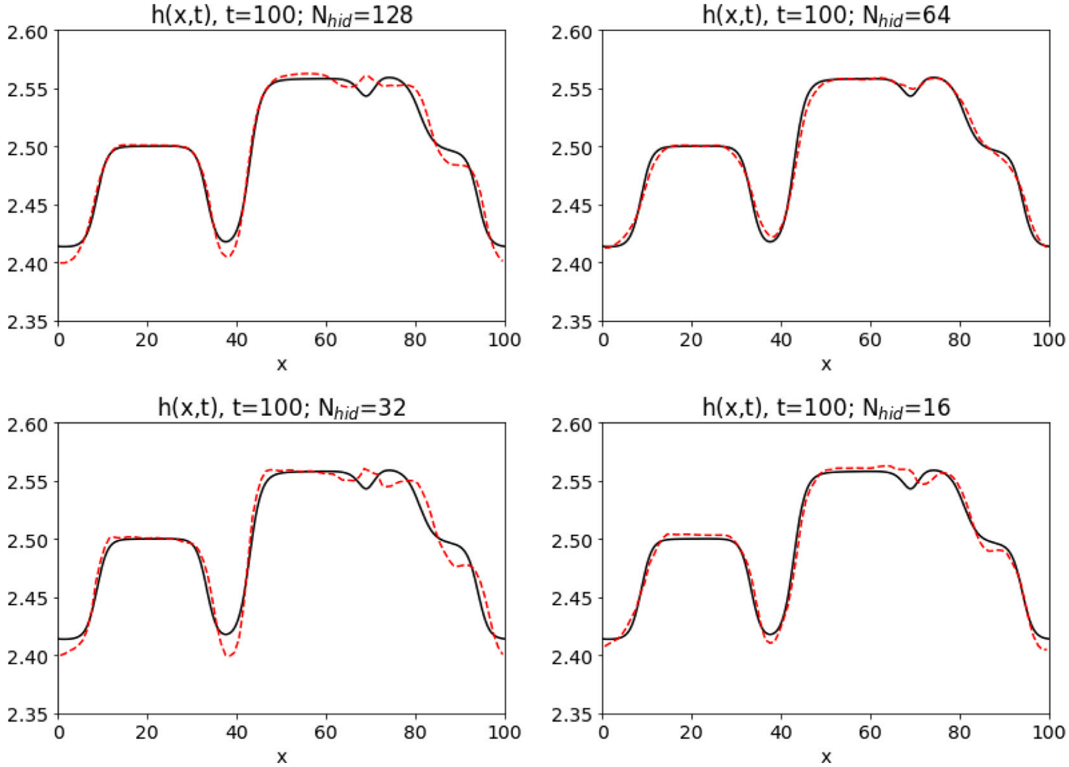


Fig. 12. Water height $h(x,t)$ at time $t = 100$ in simulations of the full model (black solid line) with $H_0 = 2.5$ and MCL-NN reduced models with $N_x = 100$ ($\Delta x = 1$) trained on $H_0 = 2$ with $n = 200$ and $T^{train} = 80$ with time-step $\Delta t = 0.05$ (red dashed line). Top left, top right, bottom left, bottom right — $N_{hid} = 128, 64, 32, 16$, respectively. Initial conditions with $H_0 = 2.5$ and other parameters as in #2 in Table 1. Compare with the top right plot in Fig. 7 where $H_0 = 2$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

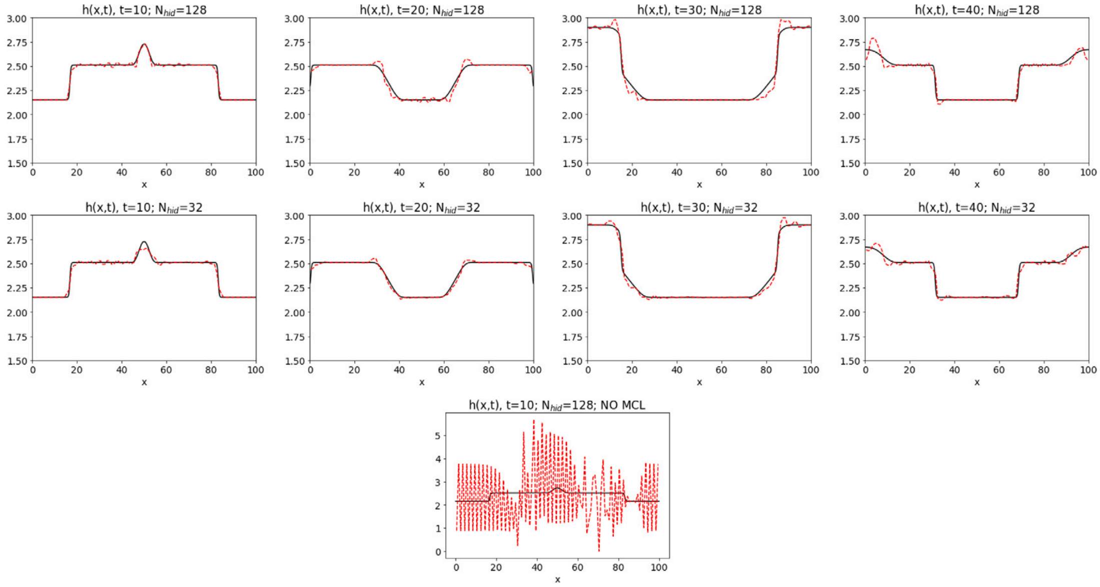


Fig. 13. Water height $h(x,t)$ at time $t = 10$, 20, 30, 40 in simulations of the full model (black solid line) with $H_0 = 2.5$ and MCL-NN reduced models with $N_x = 100$ ($\Delta x = 1$) trained on $H_0 = 2$ with $n = 200$ and $T^{train} = 80$ with time-step $\Delta t = 0.05$ (red dashed line). Top row — $N_{hid} = 128$, middle row — $N_{hid} = 32$, respectively. Bottom row — example of $h(x,t)$ at $t = 10$ in simulations of the subgrid model with $N_{hid} = 128$ without the MCL fix. Initial conditions with $H_0 = 2.4$ and $a = 0.5$ in (32). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

However, when the NN reduced model is not trained sufficiently well, the flux limiter plays a crucial role in suppressing non-physical oscillations. Thus, the MCL can be used to alleviate the problem of insufficient data and correct NN subgrid parametrizations when they produce non-physical results. The computational complexity of the NN- and MCL-NN reduced models is approximately the same since both of them are implemented on a coarse grid. Therefore, a detailed comparison of the numerical results generated by these two models can be used as an indicator of whether the machine-learning model is trained sufficiently well. Such an a posteriori test can then be used to assess the training quality of the neural network and make decisions regarding further training. Moreover, the flux limiter becomes crucial when the NN reduced models are utilized in simulations with non-smooth initial conditions. This is consistent with the general theory of numerical methods for hyperbolic problems. Since the NN subgrid model aims to represent high-order (in space) flux approximations, flux limiters have to be utilized for non-smooth solutions.

CRediT authorship contribution statement

Ilya Timofeyev: Writing – original draft, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization. **Alexey Schwarzmann:** Writing – original draft, Validation, Software, Methodology, Investigation. **Dmitri Kuzmin:** Writing – review & editing, Project administration, Methodology, Funding acquisition, Conceptualization.

Acknowledgments

I.T. was partially supported by the grant ONR N00014-17-1-2845. The work of A.S. was supported by the German Academic Scholarship Foundation (Studienstiftung des deutschen Volkes). D.K. acknowledges the financial support by the German Research Foundation (DFG) under grant KU 1530/30-1. We also would like to thank anonymous referees for their valuable suggestions that lead to a considerable improvement of our paper.

Appendix. Running time

The additional information provided in this appendix gives a rough idea about the running times of different models. The code that we used for SWE simulations is implemented in Python. It was executed on a 56-core Intel(R) Xeon(R) CPU E5-2690 v4 2.60 GHz Linux machine. The running times for a single simulation with $T = 500$ are presented in Table A.2. These results take CPU utilization into account and are equivalent to a single-CPU run for all models. The wall-time for MCL-NN128 and MCL-NN64 is approximately equivalent to the running times of MCL-NN32 and MCL-NN16. Simulations using the MCL-NN128 and MCL-NN64 models are automatically parallelized by PyTorch. This parallelization might introduce substantial additional overhead.

Table A.2
Running time (seconds) for a single simulation with $T = 500$ of the direct solver (LLF) with $N_f = 2000$ points in space and time-step $\Delta t = 0.005$ and MCL-NN models with $N_f = 100$ points in space and $\Delta t = 0.05$. PyTorch automatically parallelizes simulations of MCL-NN128 and MCL-NN64 models with CPU utilization 957% and 991%, respectively. Simulations of MCL-NN32 and MCL-NN16 are not parallelized.

LLF	MCL-NN128	MCL-NN64	MCL-NN32	MCL-NN16
493	1943	1857	177	165

Simulations with MCL-NN32 are approximately 2.7 times faster than simulations using the fully-resolved model (LLF). Therefore, a substantial acceleration of the MCL-NN reduced models can be achieved by using a much larger time-step, compared with the direct numerical simulation. This speedup is a direct consequence of using a much coarser computational mesh in simulations with reduced models.

References

[1] J. Smagorinsky, General circulation experiments with the primitive equations: I. The basic experiment, Mon. Weather Rev. 91 (3) (1963) 99–164, http://dx.doi.org/10.1175/1520-0493(1963)091<0099:GCEWTP>2.3.CO;2, URL https://journals.ametsoc.org/view/journals/mwre/91/3/1520-0493_1963_091_0099_gcewtp_2_3_co_2.xml.

[2] C. Leith, Stochastic models of chaotic systems, Phys. D Nonlinear Phenom. 98 (2) (1996) 481–491, http://dx.doi.org/10.1016/0167-2789(96)00107-8, URL https://www.sciencedirect.com/science/article/pii/0167278996001078, Nonlinear Phenomena in Ocean Dynamics.

[3] A.J. Majda, I. Timofeyev, E. Vanden-Eijnden, A mathematics framework for stochastic climate models, Comm. Pure Appl. Math. 54 (2001) 891–974.

[4] J. Berner, G.J. Shutts, M. Leutbecher, T.N. Palmer, A spectral stochastic kinetic energy backscatter scheme and its impact on flow-dependent predictability in the ECMWF ensemble prediction system, J. Atmos. Sci. 66 (3) (2009) 603–626, http://dx.doi.org/10.1175/2008JAS2677.1, URL https://journals.ametsoc.org/view/journals/atsc/66/3/2008jas2677.1.xml.

[5] M.F. Jansen, I.M. Held, Parameterizing subgrid-scale eddy effects using energetically consistent backscatter, Ocean. Model. 80 (2014) 36–48, http://dx.doi.org/10.1016/j.ocemod.2014.06.002, URL https://www.sciencedirect.com/science/article/pii/S1463500314000766.

[6] S.I. Dolaptchiev, U. Achatz, I. Timofeyev, Stochastic closure for local averages in the finite-difference discretization of the forced Burgers equation, Theor. Comput. Fluid Dyn. 27 (3-4) (2013) 297–317.

[7] M. Zacharuk, S.I. Dolaptchiev, U. Achatz, I. Timofeyev, Stochastic subgrid-scale parametrization for one-dimensional shallow-water dynamics using stochastic mode reduction, Q. J. R. Meteorol. Soc. 144 (715) (2018) 1975–1990, http://dx.doi.org/10.1002/qj.3396, arXiv:https://rmets.onlinelibrary.wiley.com/doi/pdf/10.1002/qj.3396.

[8] J. Berner, U. Achatz, L. Batte, L. Bengtsson, A. de la Camara, H.M. Christensen, M. Colangeli, D.R.B. Coleman, D. Crommelin, S.I. Dolaptchiev, C.L.E. Franzke, P. Friederichs, P. Imkeller, H. Jarvinen, S. Juricke, V. Kitsios, F. Lott, V. Lucarini, S. Mahajan, T.N. Palmer, C. Penland, M. Sakradzija, J.-S. von Storch, A. Weisheimer, M. Weniger, P.D. Williams, J.-I. Yano, Stochastic parameterization: Toward a new view of weather and climate models, Bull. Am. Meteorol. Soc. 98 (3) (2017) 565–588, http://dx.doi.org/10.1175/BAMS-D-15-00268.1, URL https://journals.ametsoc.org/view/journals/bams/98/3/bams-d-15-00268.1.xml.

[9] V. Resseguier, M. Ladvig, D. Heitz, Real-time estimation and prediction of unsteady flows using reduced-order models coupled with few measurements, J. Comput. Phys. 471 (2022) 111631, http://dx.doi.org/10.1016/j.jcp.2022.111631, URL https://www.sciencedirect.com/science/article/pii/S0021999122006945.

[10] U. Achatz, G. Branstator, A two-layer model with empirical linear corrections and reduced order for studies of internal climate variability, J. Atmos. Sci. 56 (1999) 3140–3160.

[11] M. Germano, U. Piomelli, P. Moin, W.H. Cabot, A dynamic subgrid-scale eddy viscosity model, Phys. Fluids A: Fluid Dyn. 3 (7) (1991) 1760–1765, http://dx.doi.org/10.1063/1.857955, arXiv:https://pubs.aip.org/aip/pof/article-pdf/3/7/1760/12459782/1760_1_online.pdf.

[12] D.K. Lilly, A proposed modification of the Germano subgrid-scale closure method, Phys. Fluids A Fluid Dyn. 4 (3) (1992) 633–635, http://dx.doi.org/10.1063/1.858280, arXiv:https://pubs.aip.org/aip/pof/article-pdf/4/3/633/12443162/633_1_online.pdf.

[13] P. Moin, K. Squires, W. Cabot, S. Lee, A dynamic subgrid-scale model for compressible turbulence and scalar transport, Phys. Fluids A: Fluid Dyn. 3 (11) (1991) 2746–2757.

[14] D.D. Holm, B.T. Nadiga, Modeling mesoscale turbulence in the barotropic double-gyre circulation, J. Phys. Oceanogr. 33 (11) (2003) 2355–2365.

[15] T. Bolton, L. Zanna, Applications of deep learning to ocean data inference and subgrid parameterization, J. Adv. Model. Earth Syst. 11 (2019) 376–399.

[16] J. Alcala, I. Timofeyev, Subgrid-scale parametrization of unresolved scales in forced Burgers equation using generative adversarial networks (GAN), Theor. Comput. Fluid Dyn. 35 (2021) 875–894.

[17] C. Zhang, P. Perezhogin, C. Gultekin, A. Adcroft, C. Fernandez-Granda, L. Zanna, Implementation and evaluation of a machine learned mesoscale eddy parameterization into a numerical ocean circulation model, J. Adv. Model. Earth Syst. 15 (10) (2023) http://dx.doi.org/10.1029/2023MS003697, e2023MS003697, arXiv:https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2023MS003697.

[18] N. Agarwal, D. Kondrashov, P. Dueben, E. Ryzhov, P. Berloff, A comparison of data-driven approaches to build low-dimensional ocean models, J. Adv. Model. Earth Syst. 13 (9) (2021) http://dx.doi.org/10.1029/2021MS002537, e2021MS002537, arXiv:https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2021MS002537.

[19] M. Lino, S. Fotiadis, A.A. Bharath, C.D. Cantwell, Current and emerging deep-learning methods for the simulation of fluid dynamics, Proc. R. Soc. A: Math. Phys. Eng. Sci. 479 (2275) (2023) 20230058, http://dx.doi.org/10.1098/rspa.2023.0058, arXiv:https://royalsocietypublishing.org/doi/pdf/10.1098/rspa.2023.0058.

[20] A.d. Saint-Venant, et al., Theorie du mouvement non permanent des eaux, avec application aux crues des rivieres et a l'introduction de marees dans leurs lits, Comptes Rendus Des Seances de L' Acad. Des Sci. 36 (1871) 154–174.

[21] D. Kuzmin, Monolithic convex limiting for continuous finite element discretizations of hyperbolic conservation laws, Comput. Methods Appl. Mech. Engrg. 361 (2020) 112804.

[22] J.-L. Guermond, M. Nazarov, B. Popov, I. Tomas, Second-order invariant domain preserving approximation of the Euler equations using convex limiting, SIAM J. Sci. Comput. 40 (5) (2018) A3211–A3239.

[23] R.J. LeVeque, Finite Volume Methods for Hyperbolic Problems, vol. 31, Cambridge University Press, 2002.

[24] C.-W. Shu, S. Osher, Efficient implementation of essentially non-oscillatory shock-capturing schemes, J. Comput. Phys. 77 (2) (1988) 439–471.

[25] S. Gottlieb, C.-W. Shu, E. Tadmor, Strong stability-preserving high-order time discretization methods, SIAM Rev. 43 (1) (2001) 89–112.

[26] S. Gottlieb, D. Ketcheson, C.-W. Shu, Strong Stability Preserving Runge-Kutta and Multistep Time Discretizations, World Scientific, 2011.

[27] A. Schwarzmann, Discrete model reduction for the shallow water equations using neural networks and flux limiting, 2023.

[28] J.S. Alcala, Subgrid-scale parametrization of unresolved processes (Ph.D. thesis), 2021.

[29] S.I. Dolaptchiev, I. Timofeyev, U. Achatz, Subgrid-scale closure for the inviscid Burgers-Hopf equation, Comm. Math. Sci. 11 (3) (2013) 757–777.

[30] A.L. Maas, A.Y. Hannun, A.Y. Ng, et al., Rectifier nonlinearities improve neural network acoustic models, in: Proc. ICML, 30, (1) Atlanta, GA, 2013, p. 3.

[31] J.-L. Guermond, B. Popov, Invariant domains and first-order continuous finite element approximation for hyperbolic systems, SIAM J. Numer. Anal. 54 (4) (2016) 2466–2489.

[32] A. Harten, P.D. Lax, B. van Leer, On upstream differencing and godunov-type schemes for hyperbolic conservation laws, SIAM Rev. 25 (1) (1983) 35–61, http://dx.doi.org/10.1137/1025002.

[33] H. Hajduk, Algebraically constrained finite element methods for hyperbolic problems with applications in geophysics and gas dynamics (Ph.D. thesis), Dortmund, Technische Universität, 2022.

[34] J.P. Boris, D.L. Book, Flux-corrected transport. I. SHASTA, a fluid transport algorithm that works, J. Comput. Phys. 11 (1) (1973) 38–69, http://dx.doi.org/10.1016/0021-9991(73)90147-2.

[35] S.T. Zalesak, Fully multidimensional flux-corrected transport algorithms for fluids, J. Comput. Phys. 31 (3) (1979) 335–362, http://dx.doi.org/10.1016/0021-9991(79)90051-2.

[36] D. Kuzmin, M. Möller, J.N. Shadid, M. Shashkov, Failsafe flux limiting and constrained data projections for equations of gas dynamics, J. Comput. Phys. 229 (23) (2010) 8766–8779, http://dx.doi.org/10.1016/j.jcp.2010.08.009.

[37] R. Löhner, K. Morgan, J. Peraire, M. Vahdati, Finite element flux-corrected transport (FEM–FCT) for the euler and Navier–Stokes equations, Internat. J. Numer. Methods Fluids 7 (10) (1987) 1093–1109.

[38] D. Kuzmin, H. Hajduk, Property-Preserving Numerical Schemes for Conservation Laws, World Scientific, 2023.

[39] H. Hajduk, D. Kuzmin, Bound-preserving and entropy-stable algebraic flux correction schemes for the shallow water equations with topography, 2022, ArXiv Preprint, arXiv:2207.07261.

[40] P. Azerad, J.-L. Guermond, B. Popov, Well-balanced second-order approximation of the shallow water equation with continuous finite elements, SIAM J. Numer. Anal. 55 (6) (2017) 3203–3224, http://dx.doi.org/10.1137/17M1122463.

[41] C. Lohmann, D. Kuzmin, Synchronized flux limiting for gas dynamics variables, J. Comput. Phys. 326 (2016) 973–990.

[42] W. Pazner, Sparse invariant domain preserving discontinuous Galerkin methods with subcell convex limiting, Comput. Methods Appl. Mech. Engrg. 382 (2021) 113876.

[43] A. Schwarzmann, clf1: A collection of finite elements, finite volume, and machine learning solvers for one-dimensional conservation law problems; URL https://github.com/A-dot-S-dot/cls1.

[44] D. Drikakis, W. Rider, High-Resolution Methods for Incompressible and Low-Speed Flows, Springer, 2005.

[45] F.F. Grinstein, C. Fureby, On monotonically integrated large eddy simulation of turbulent flows based on FCT algorithms, in: D. Kuzmin, R. Löhner, S. Turek (Eds.), Flux-Corrected Transport: Principles, Algorithms, and Applications, Springer, 2012, pp. 67–90.

[46] F.F. Grinstein, L.G. Margolin, W.J. Rider, Implicit Large Eddy Simulation, Cambridge University Press, 2007.